COS 214 - P3

Rei Piater-Boswell (u25678592)
Leanne van der Horst (u24566935)
Declan McCullough (u21760022)

if(Bottle.contains(water)){Bottle.replace(alcohol)}

Event name & description:

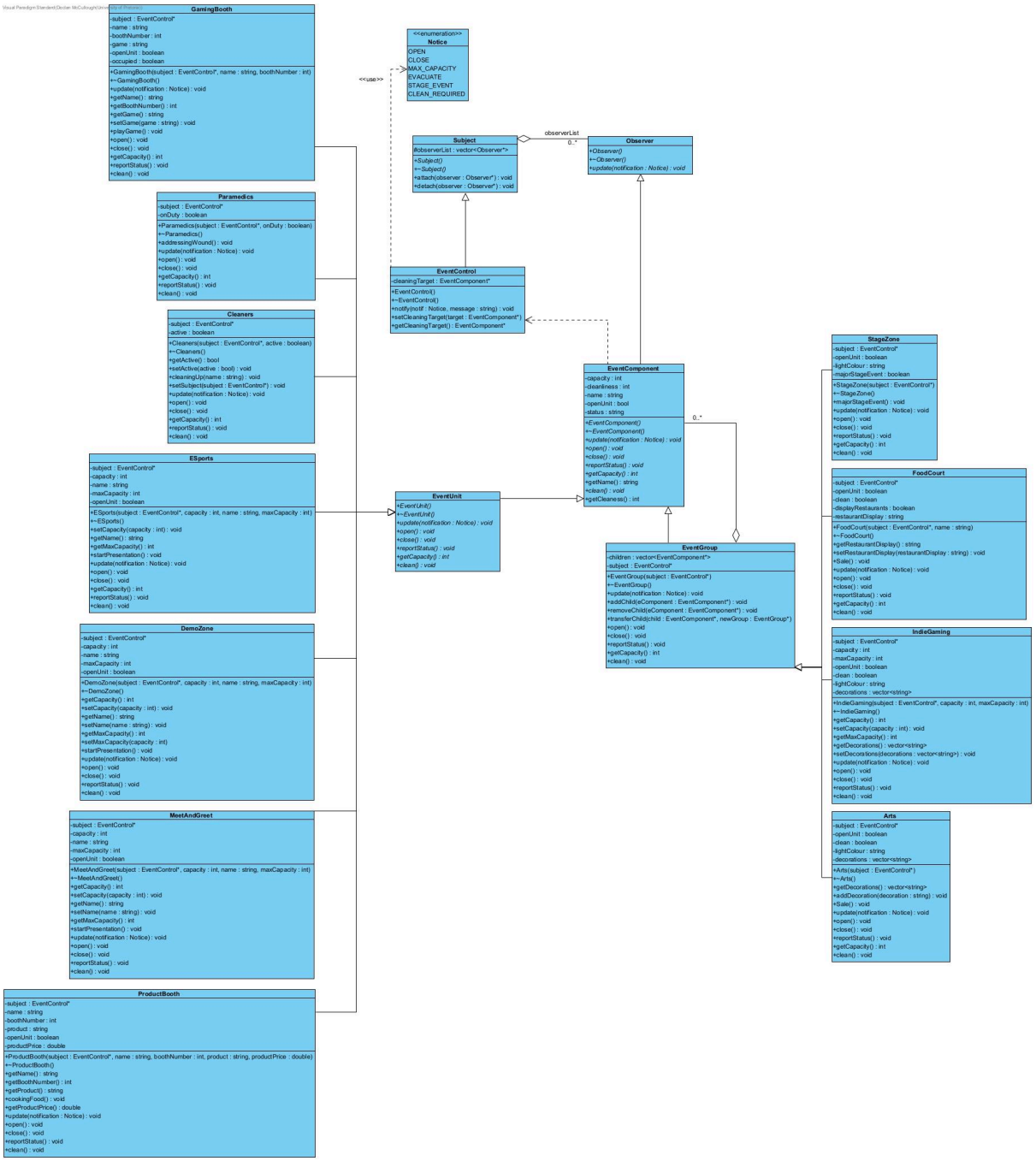
GameEvents R Us, for all your gaming convention planning needs. We plan gaming conventions, allow for the adding of multiple zones and events within the zones. Notifications can be sent to zones and all events within said zone, keeping everyone you wish informed on current events! Also allows cleaners to be sent to different zones and restore cleanliness if cleanliness reaches 0! Keeping all those stinky Gamerzs clean clean clean!!

five concrete kinds of event units : Cleaners, are sent around and clean areas if their cleanliness are equal to 0.
Paramedics are put on duty when the event opens and are actively on standby.
ProductBooths are Booths that when notified of opening, they start their sales on the product they are selling and start cooking food.
The Esports unit is where presentations will be given. Presentations start and close with open and close notifications.

Demo zones will also have the same presentation functionality. MeetandGreet units will also have presentations for meeting celebrities of the gaming nature.

three kinds of grouping/area: FoodCourt would be a zone that would mostly host ProductBooths that sell food.
StageZone is a zone that houses units where presentations happen such as demos.
The art zone would mostly house units such as productBooths with products of type art and can add decorations to said units.

UML class diagram:



Participant mapping:

Composite Participants:

EventComponent: Component

StageZone, FoodCourt, IndieGaming and Arts: Composites

GamingBooth, Paramedics, Cleaners, ESports, DemoZone, MeetAndGreet, and ProductBooth: Leaves

Observer Participants:

Subject: Abstract Subject
EventControl and Composites: Concrete Subject
Observer: Abstract Observer
EventComponent, Leaves, and Composites: Concrete Observer

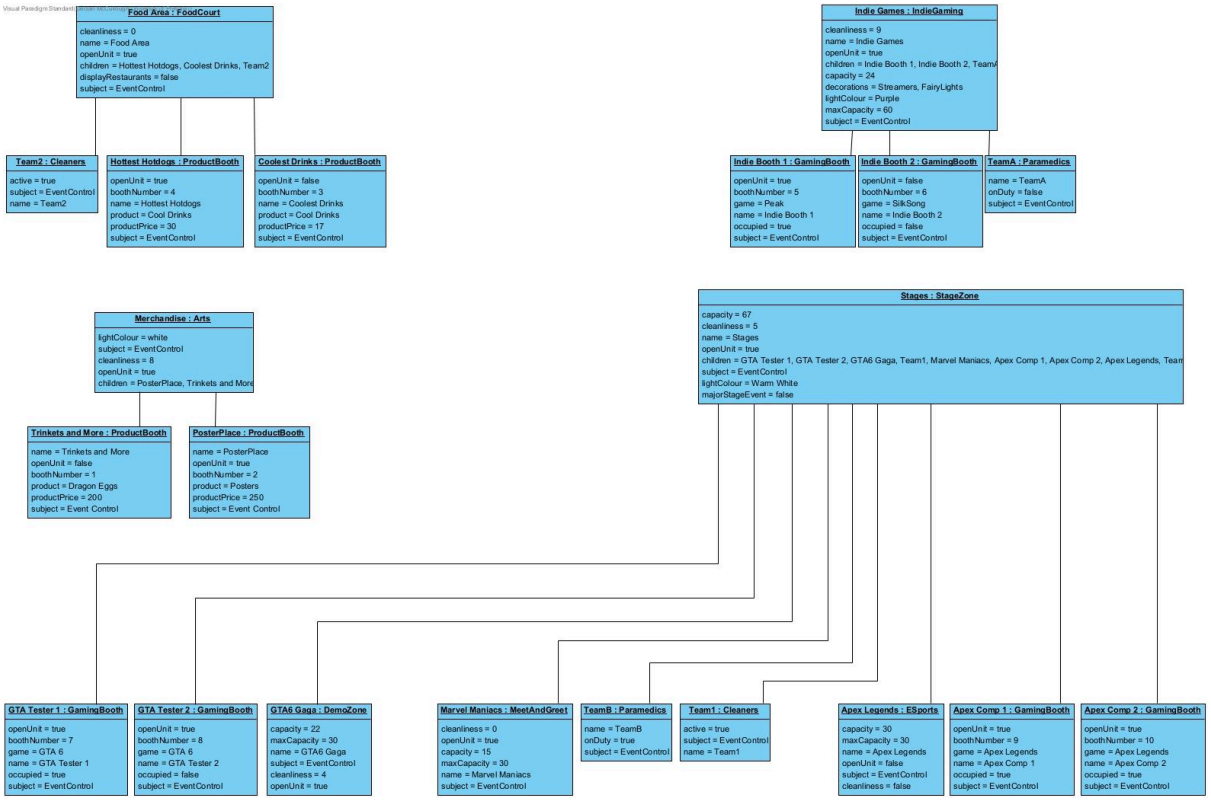
Reason for multiple participants:

The reason for the Composites being Observers as well as Subjects is that they may be part of a hierarchy of a different zone and need to keep track of the change in state in said hierarchy. The reason for them also being Subjects is that their states can change and their observers, the leaves or other composites, need to keep track of the changes and act accordingly. This is seen in the ESports leaf that is observing its parent Stage composite and can, for example, cause all the people that are sitting in the seating area to leave if an EVACUATE message is sent, or even if the state of the Stage composite is changed to close.

Design rationale:

a) Our Composite is a genuine part-whole tree, because the different composites and leave, which have their own functionality on their own - granted it is quite minimal, can be combined into something where they all work together allowing for more functionality to be made between the different objects. This then creates a whole new “object” since all the different parts are combined to create something new and functional.
b) The Observer pattern is required for the functionality of the program, because there could be a lot of state changes that happened to a subject and could cause some hard coded calls to not properly function since they may not be able to access certain data that could have changed. This is also because it would allow for the program to run more efficiently since the calls are being handled by an external object rather than the object itself.
c) The composites would be the objects that can contain the event objects as they are the leaves to the Composite pattern.
d) The Subject is the one that owns references to observers in order to push updates to them.
e) The reason why an object can be both an observer and a subject without being a misuse of either pattern is because an object can be owned by a parent whilst being a parent to children objects itself. This allows for the object to observe its parent and act according to the state changes and even send those changes down to its children to allow for them to react to something that is affecting their parent.
Our Observer pattern uses push to send notifications down to observers. This would allow us to send one notification from the EventControl to all the observers and their children that are observing the EventControl object. This would then allow for low coupling to occur as we do not need to try and send an update from the EventControl object to the observers.

Composite object diagram:

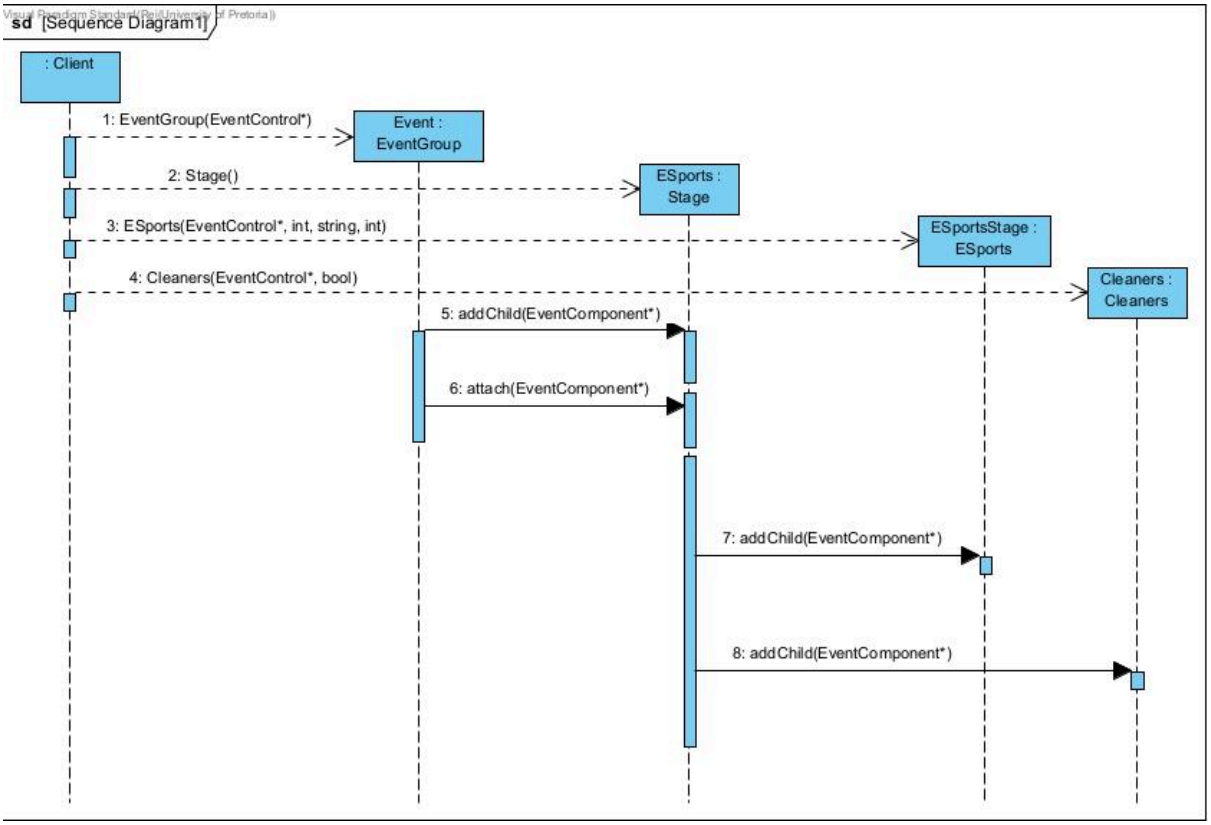


Event rules and original features:

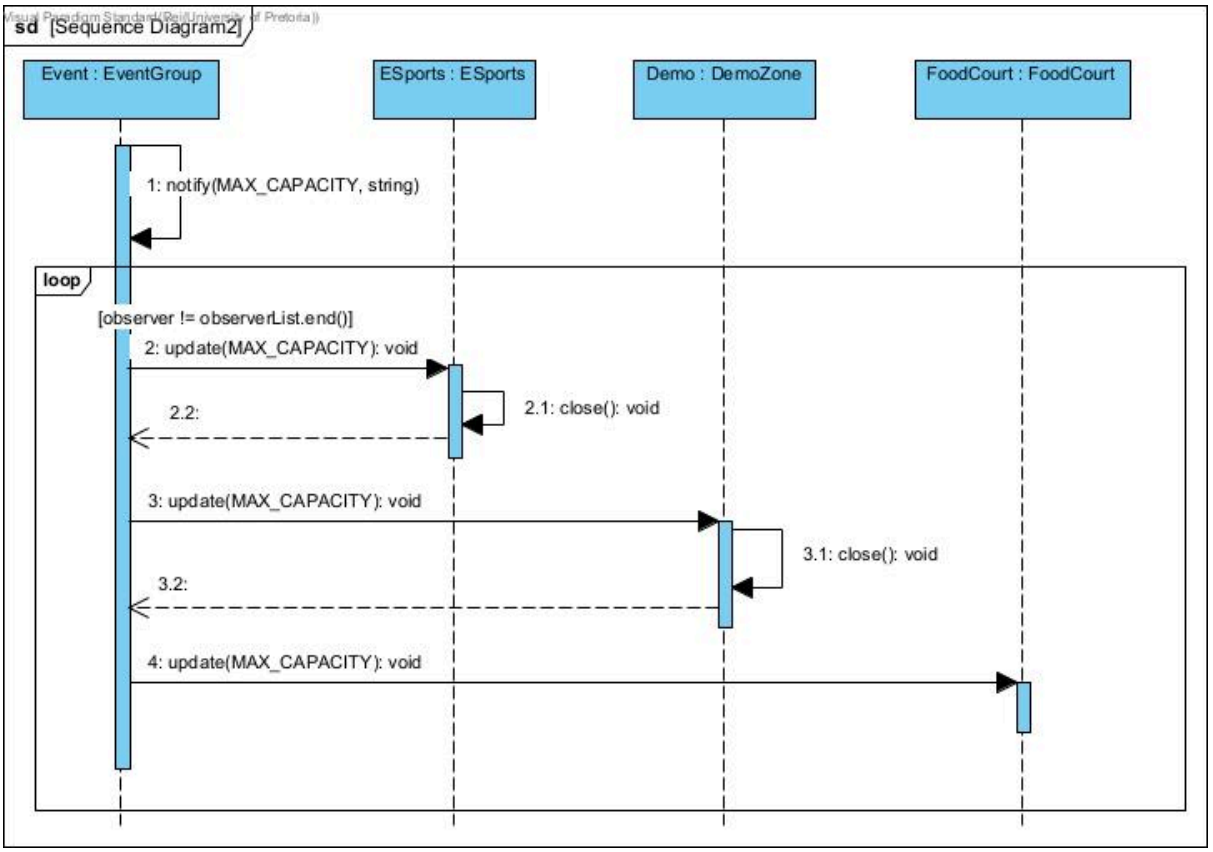
OPEN: Sets a zone or individual objects, such as a booth, to be seen as open. If it is already open then nothing happens. For example, this causes a stage to be able to receive more people adding to its capacity and be interacted with.
CLOSE: Sets a zone or individual objects, such as a booth, to be seen as closed causing it to not be active, for example a stage loses its functionality and has no capacity.
MAX_CAPACITY: if a zone has a capacity and the current capacity of the zone exceeds its set max capacity then the zone would no longer accept more people and “kick” some people out to go down to the max capacity. However, if a zone does not keep track of its capacity then it would do nothing different - continue as normal.
EVACUATE: Sets all zones to be closed, clears everything’s capacities causing everything to be seen as closed and cleared of people.
STAGE_EVENT: If the zone is a Stage Zone then it would cause all the different stages to combine into one zone, adding their max capacities together and acting as one stage. If a zone is not a Stage Zone then it would continue as normal since it is not a stage.

Sequence Diagrams:

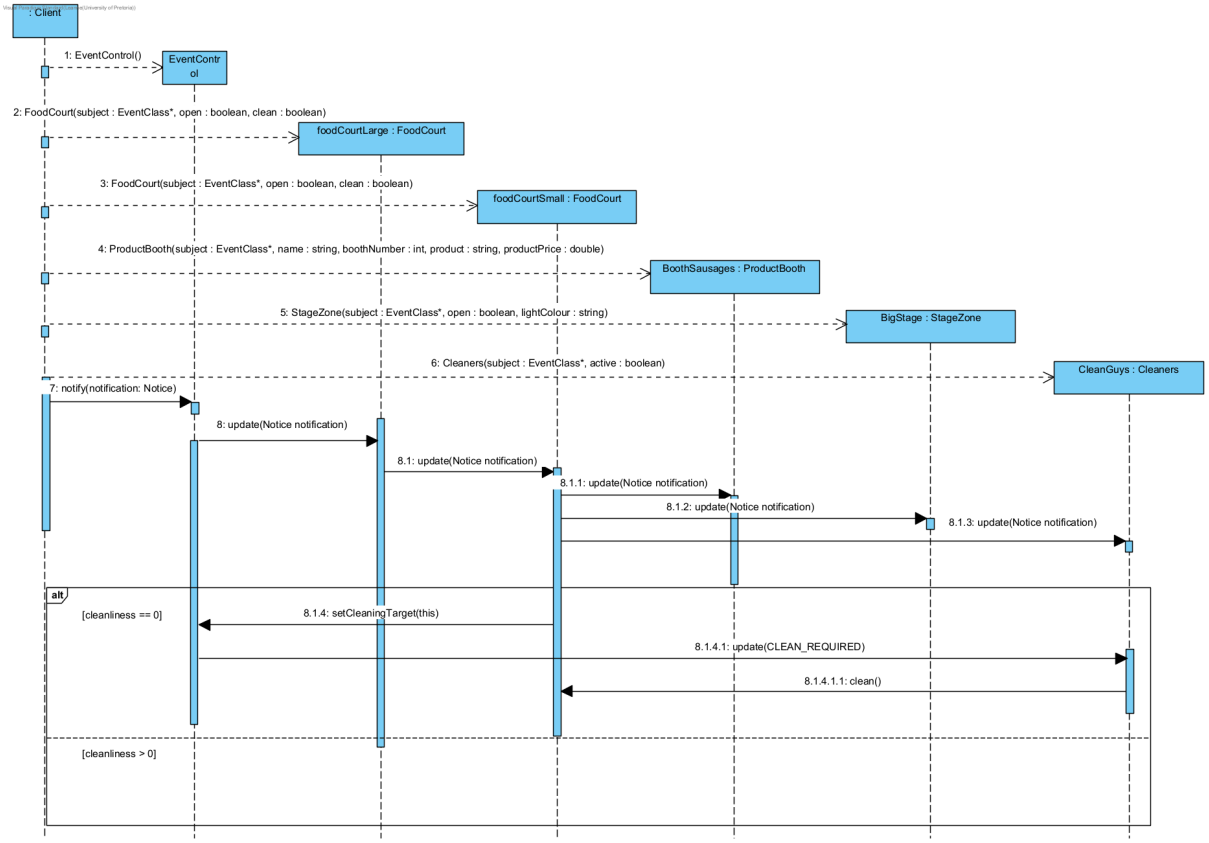
SD1


sd [Sequence Diagram1]
: Client
Event : EventGroup
ESports : Stage
ESportsStage : ESports
Cleaners : Cleaners
1: EventGroup(EventControl*)
2: Stage()
3: ESports(EventControl*, int, string, int)
4: Cleaners(EventControl*, bool)
5: addChild(EventComponent*)
6: attach(EventComponent*)
7: addChild(EventComponent*)
8: addChild(EventComponent*)

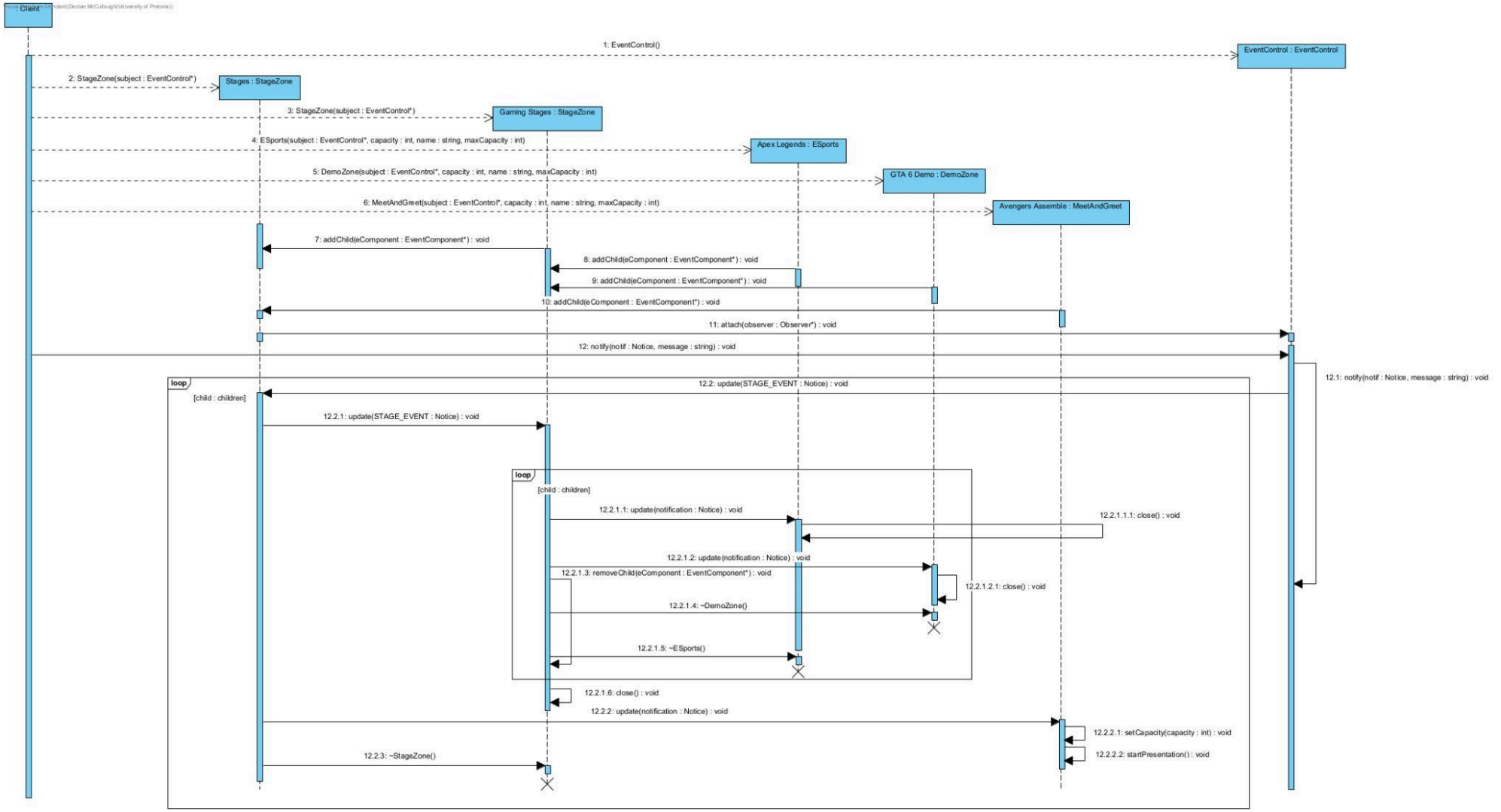
SD2


sd [Sequence Diagram2]
Event : EventGroup
ESports : ESports
Demo : DemoZone
FoodCourt : FoodCourt
1: notify(MAX_CAPACITY, string)
loop
[observer != observerList.end()]
2: update(MAX_CAPACITY): void
2.1: close(): void
2.2:
3: update(MAX_CAPACITY): void
3.1: close(): void
3.2:
4: update(MAX_CAPACITY): void

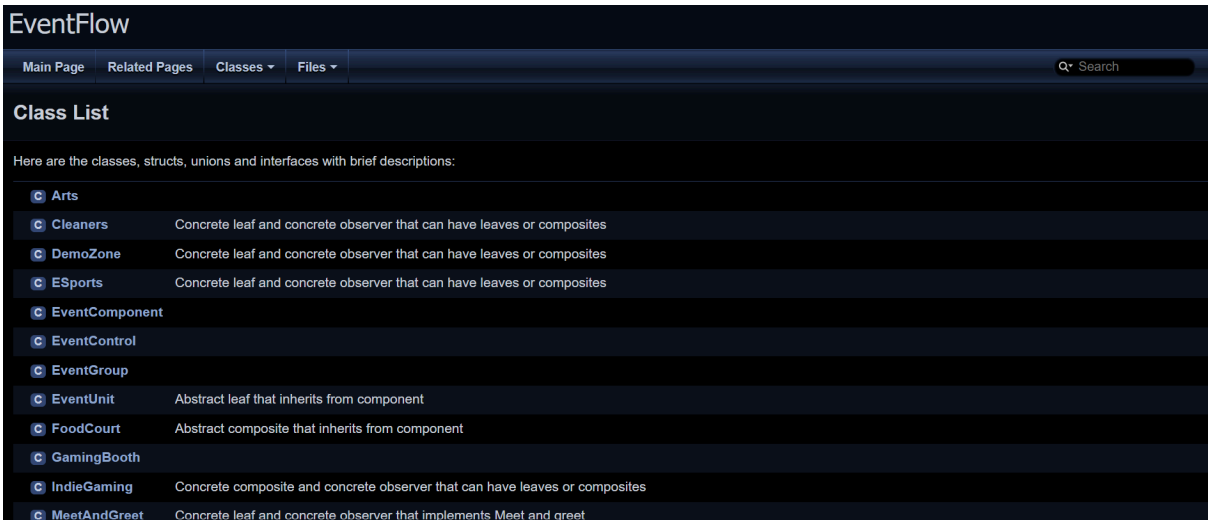
SD3


: Client
1: EventControl()
EventContr
ol
2: FoodCourt(subject : EventClass*, open : boolean, clean : boolean)
foodCourtLarge : FoodCourt
3: FoodCourt(subject : EventClass*, open : boolean, clean : boolean)
foodCourtSmall : FoodCourt
4: ProductBooth(subject : EventClass*, name : string, boothNumber : int, product : string, productPrice : double)
BoothSausages : ProductBooth
5: StageZone(subject : EventClass*, open : boolean, lightColour : string)
BigStage : StageZone
6: Cleaners(subject : EventClass*, active : boolean)
CleanGuys : Cleaners
7: notify(notification: Notice)
8: update(Notice notification)
8.1: update(Notice notification)
8.1.1: update(Notice notification)
8.1.2: update(Notice notification)
8.1.3: update(Notice notification)
alt
[cleanliness == 0]
8.1.4: setCleaningTarget(this)
8.1.4.1: update(CLEAN_REQUIRED)
8.1.4.1.1: clean()
[cleanliness > 0]

SD4


: Client
1: EventControl()
EventControl : EventControl
2: StageZone(subject : EventControl*)
Stages : StageZone
3: StageZone(subject : EventControl*)
Gaming Stages : StageZone
4: ESports(subject : EventControl*, capacity : int, name : string, maxCapacity : int)
Apex Legends : ESports
5: DemoZone(subject : EventControl*, capacity : int, name : string, maxCapacity : int)
GTA 6 Demo : DemoZone
6: MeetAndGreet(subject : EventControl*, capacity : int, name : string, maxCapacity : int)
Avengers Assemble : MeetAndGreet
7: addChild(eComponent : EventComponent*) : void
8: addChild(eComponent : EventComponent*) : void
9: addChild(eComponent : EventComponent*) : void
10: addChild(eComponent : EventComponent*) : void
11: attach(observer : Observer*) : void
12: notify(notif : Notice, message : string) : void
12.1: notify(notif : Notice, message : string) : void
loop
[child : children]
12.2: update(STAGE_EVENT : Notice) : void
12.2.1: update(STAGE_EVENT : Notice) : void
loop
[child : children]
12.2.1.1: update(notification : Notice) : void
12.2.1.1.1: close() : void
12.2.1.2: update(notification : Notice) : void
12.2.1.3: removeChild(eComponent : EventComponent*) : void
12.2.1.2.1: close() : void
12.2.1.4: ~DemoZone()
12.2.1.5: ~ESports()
12.2.1.6: close() : void
12.2.2: update(notification : Notice) : void
12.2.2.1: setCapacity(capacity : int) : void
12.2.2.2: startPresentation() : void
12.2.3: ~StageZone()

Doxygen evidence:



Github Repo link:

https://github.com/Lenn-lolz/COS214_P3.git

GitHub workflow reflection:

We had originally planned on dividing the work up equally amongst each other where everyone has a chance to code something to contribute to EventFlow. We started by adding the stub files of the project to the main branch so that everyone can have access to the very basic form of our header files and work from there.
Rei started with the Composite pattern by trying to implement the basic functionality of the leaves of the Composite pattern and uploaded the files to a separate branch called CompositePattern to ensure that the files in main are not overwritten without the other members wanting to merge it into the main branch.
Declan made a pull request from one branch, leaf-node-files, to the main branch making those files the base that we were working from for the duration of the project.
Leanne made a branch for her work on the EventControl, the Observer pattern, so that the changes will not overwrite the files in the main branch in case something was not working properly.
The group made commits to different branches for quite significant changes that were made to the files to prevent the files that were agreed to work relatively from being overwritten, thus losing any progress if we had to discard the changes that were done to the files in the different branches. We would then find some agreement on certain files and make sure that those were merged into the main branch.

Team contribution statement:

All three of the members contributed to the creation of the UML's design for the EventFlow project. We all got together on a Discord call where Declan streamed his point of view and all three of us gave suggestions to how we want the EventFlow

program to work. We then designed our UML around the ideas we had for the EventFlow program, resulting in the current UML that we have now. We then uploaded that and the basic stub files for the different parts of the program to our GitHub repository to allow for everyone to be able to access them and work on their own parts.

We then proceeded to work on the code for the EventFlow program where we tried to divide the work up equally between each other so that everyone can have roughly equal work, but we also worked on each other's parts if the other asked for help to fill some gaps where necessary. However, the coding was in the end mainly done by Declan and Leanne as Rei had other conflicts that prevented them from working on the code.

Both Leanne and Declan worked on the same parts of the code, practically working in parallel to each other allowing each other to fill the other's gap in knowledge on a certain concept or task. This allowed for more efficient work as they could quickly get through the coding of the EventFlow program.

Rei worked mostly on the PDF submission, typing the writing answers and ensuring that everything is in the PDF that is required of the group to include.

Everyone worked on the creation of the Sequence Diagrams (SD) where Rei worked on SD 1 and 2, Leanne worked on SD 3 and Declan worked on SD 4. Leanne also worked on the event's name and description that can be found at the top of the PDF.